

Student Management System

Architecture Overview — from the database to the browser

1. System at a Glance

A three-tier web app: a React single-page app talks to a FastAPI backend over a JSON REST API; the backend reads and writes a single relational database through the SQLAlchemy ORM, with one model class per table (see Section 3). The same codebase runs against SQLite with zero setup for local development, or PostgreSQL for a shared/production deployment.

Request/response flow

Browser	React SPA (localhost:3000)	FastAPI backend (localhost:8080)	SQLite / PostgreSQL
---------	-------------------------------	-------------------------------------	---------------------

HTTP/JSON, arrows flow both ways: the browser renders pages and calls the API; the API queries the database and returns JSON that the React components render.

Technology stack

Layer	Choice
Frontend	React 18 + React Router 6, plain CSS (no UI framework); Create React App tooling
Backend	FastAPI (Python), served by Uvicorn
Data layer	SQLAlchemy 2.0 ORM — declarative models, typed columns, relationships, Session per request
Database	SQLite file (default, auto-created) or PostgreSQL (via DATABASE_URL)
Auth	Custom HMAC-SHA256 signed session tokens + PBKDF2-SHA256 password hashing (hand-rolled, not a JWT library)

Repository layout

```
backend/    FastAPI application (main.py + app/)
frontend/   React application (Create React App, src/)
database/   PostgreSQL starter schema + demo-data SQL script
README.md   Setup instructions for both servers
```

2. Frontend Folder Structure

Create React App project. Everything lives under `frontend/src`. Each role (Admin / Teacher / Student) has its own dashboard shell, layout, and set of page components, all backed by the same shared context/services/utils layer.

Top-level files

File	Purpose
<code>index.js</code>	React DOM entry point — mounts <code><App /></code> into the page.
<code>App.js</code>	Root component: wraps the app in <code>ThemeProvider</code> + <code>AuthProvider</code> , and declares every top-level route (3 login pages + 3 role dashboards, each behind <code>AuthGuard</code>).
<code>App.css</code>	Global base styles shared across the whole app.

context/ — React Context providers (global state)

File	Purpose
<code>AuthContext.js</code>	Login/logout, session persistence (per-username in <code>localStorage</code> , per-tab in <code>sessionStorage</code>), restores a cached session on app load.
<code>ThemeContext.js</code>	Light/dark mode flag, persisted to <code>localStorage</code> , toggles a body CSS class.
<code>AcademicYearContext.js</code>	Currently-selected academic year + class list, shared across admin pages.

services/ — API client layer

File	Purpose
<code>api.js</code>	Original fetch-based API client (base URL, auth header, one method per endpoint).
<code>apiServiceSingleton.js</code>	The client actually used app-wide: a singleton wrapper adding retry/backoff, an offline request queue, connection-health tracking, and sequential request ordering on top of the same endpoints.

utils/ — cross-cutting helpers

File	Purpose
<code>logger.js</code>	Structured console logging for API calls (method, endpoint, payload, response/error, duration).
<code>performanceMonitor.js</code>	Tracks client-side performance metrics such as API call counts/timing.
<code>sequentialLoader.js</code>	A priority queue (<code>queueApiCall</code>) that serializes API requests instead of firing them all at once.
<code>sessionDebugger.js</code>	Exposes session/auth state to the browser console for manual debugging.

components/Auth/ & components/Common/ — shared UI

File(s)	Purpose
<code>Auth/AdminLogin.js</code> , <code>TeacherLogin.js</code> , <code>StudentLogin.js</code>	One login form per role, all posting to the same <code>/api/auth/login</code> .
<code>Auth/RoleSelection.js</code>	Landing screen for picking which role to log in as.
<code>Auth/AuthGuard.js</code>	Route guard used by <code>App.js</code> : redirects unauthenticated users to login, and users with the wrong role to their own login page.
<code>Common/AuthGuard.js</code>	A second, more defensive route guard (auto-logout on bad role/deactivation) — unused; <code>App.js</code> uses <code>Auth/AuthGuard.js</code> .
<code>Common/Pagination.js</code>	Reusable page-number control for list views.
<code>Common/ConnectionStatus.js</code>	Small banner reflecting backend connection health.
<code>Common/SequentialDashboard.js</code>	Wraps a dashboard's routes with a loading state tied to the sequential request queue.
<code>Common/ReactivationNotifications.js</code>	Notifies a user when a previously deactivated account is reactivated.
<code>Common/ThemeToggle.js</code>	Light/dark mode switch UI, backed by <code>ThemeContext</code> .

components/Admin/, Teacher/, Student/ — role dashboards

Each role folder follows the same pattern: a `Dashboard` component declares nested routes, a `Layout` provides sidebar/header chrome, and `pages/` holds one component per screen — named after the backend resource it manages (`StudentManagement.js` does CRUD on `/api/students`; `StudentSummary.js` is its read-only overview).

Role	Shell files	Pages
Admin	<code>AdminDashboard.js</code> , <code>AdminLayout.js</code>	17 pages: <code>Fee/Class/Attendance/Marks/Teacher/Student/Exam/Holiday/Schedule/Time</code> management screens, plus <code>*Summary</code> dashboard variants and <code>SystemConfiguration</code> .
Teacher	<code>TeacherDashboard.js</code> , <code>TeacherLayout.js</code>	6 pages: <code>TeacherProfile</code> , <code>AttendanceManagement</code> , <code>FeeInquiry</code> , <code>MarksManagement</code> , <code>SyllabusManagement</code> , <code>TeacherTimetable</code> .
Student	<code>StudentDashboard.js</code> , <code>StudentLayout.js</code>	6 pages: <code>PersonalDetails</code> , <code>StudentTimetable</code> , <code>AttendanceTracker</code> , <code>FeeHistory</code> , <code>AcademicPerformance</code> , <code>SyllabusProgress</code> .

3. Backend Data Layer — the ORM (backend/app/)

Every table is a SQLAlchemy 2.0 declarative model: typed Mapped[...] columns, foreign keys and relationships, grouped by purpose under app/models/. Routes query through a Session and never write SQL by hand, so types (dates, decimals, JSON, booleans) are converted by the ORM rather than by each caller.

Backend folder structure

backend/main.py App assembly: CORS, router registration, error handler, startup
backend/app/config/ Environment settings (DATABASE_URL, PORT, JWT_SECRET, CORS origins)
backend/app/database/ Engine + Session factory (session.py) and the demo-data seeder (seed.py)
backend/app/models/ The ORM: base.py, user.py, academics.py, records.py, timetable.py
backend/app/schemas/ Pydantic request-body models
backend/app/security/ Password hashing, session tokens, the auth dependency
backend/app/routers/ The HTTP layer, one module per endpoint group

Key building blocks

Name	Role
models/base.py	Declarative Base plus the ApiModel mixin: to_dict()/apply() convert between snake_case attributes and the camelCase JSON the React app speaks.
database/session.py	One engine from DATABASE_URL; get_session() is the per-request Session dependency; create_tables() builds the schema from the models.
database/seed.py	Seeds the demo school into whichever tables are still empty (safe to run on every startup).
Model.columns()	The mapped columns of a table, so writes can safely ignore fields a table doesn't have.
Model.apply(payload)	Assigns a request body onto a row, coercing strings to date/decimal/bool per column type.
Relationships	Student.school_class, SchoolClass.students, Attendance.student, ... for navigating without manual joins.

Tables

Table	What it stores
users	Login accounts for all three roles; password is a PBKDF2 hash, not plaintext.
classes	A grade/section for one academic year (e.g. Grade 10 - A).
students	Student profile + which class they belong to; primary key doubles as their username.
teachers	Teacher profile, subjects/classes assigned (stored as JSON text), class-teacher assignment.
subjects	Subject catalogue (code, name, applicable grade, max marks per exam).
attendance	One row per student/date/period: present, absent, late or excused.
marks	One row per student/subject/exam: score, max score, letter grade.
fees	One row per student/fee type: amount due, amount paid, running balance.
syllabus	One row per class/subject unit: completion status/percentage and topic list.
schedule_data	The recurring weekly timetable: one row per class/weekday/period.
calendar_grid	What a class is doing on a given date: working day, named holiday, or exam session.

SQLite vs. PostgreSQL

By default DATABASE_URL is unset, so settings.py points at a local student_management.db SQLite file, auto-created and auto-seeded on first run (zero setup). Pointing DATABASE_URL at a Postgres instance switches the same code to Postgres; database/postgresql_starter.sql creates the equivalent schema (with the CHECK constraints and indexes the ORM doesn't declare) plus the same demo dataset. Columns are declared lowercase because Postgres folds unquoted identifiers; SQLite matches identifiers case-insensitively, so one set of models serves both.

Authentication (app/security/)

passwords.py hashes with PBKDF2-HMAC-SHA256 (210,000 iterations). tokens.py mints a hand-rolled, HMAC-SHA256-signed base64url session token valid for 24 hours — functionally similar to a JWT, but not a standards-compliant one. dependencies.py's current_user is the Depends() every protected route uses: it verifies the signature and expiry, then loads the user and rejects deactivated accounts.

4. Backend Routers (backend/app/routers/)

main.py assembles the FastAPI app: CORS for the frontend origin(s), the six routers below, a lifespan hook that creates tables and seeds demo data, and a global exception handler that turns every raised HTTPException into a consistent {success, error, message, timestamp} JSON body. Registration order matters: resources.py ends in catch-all /api/{resource} routes, so every more specific router is registered before it.

health.py — liveness checks (no auth)

GET /health and GET /api/health — the same check at the root and under the API prefix, for uptime monitors and the connection indicator.

auth.py — login/session (prefix /api/auth)

Endpoint	Description
POST /login	Verify credentials, stamp lastLogin, return a signed session token + profile.
POST /register	Create a user account (default role: student) and log it in immediately.
GET /profile	Return the caller's own user record, resolved from their bearer token.
POST /logout	No-op — tokens are stateless and simply expire (24h).

resources.py — generic CRUD (prefix /api, auth required)

One RESOURCES map drives the same handlers for all 9 record tables. List responses carry the rows twice — under "data" and under the resource's own name — as different pages read different keys:

Endpoint	Description
GET /{resource}/{id}	List with query-param filtering (classId, studentId, date range, ...) and paging, or fetch one row.
POST /{resource}	Create a row (users excluded — accounts go through /api/auth/register).
PUT /{resource}/{id}	Partial update; a fee's balance is always recomputed server-side.
DELETE /{resource}/{id}	Delete by primary key; returns 409 if other records still reference it.
GET /marks fees attendance/student/{id}, /students/grade/{grade}	Convenience filters: one student's records, or every student in a grade.
POST /fees/{id}/payment	Apply a payment, recomputing amountPaid/balance server-side.
PUT /syllabus/{id}/progress, /users/{username}/activate	Tick sub-topics off (percentage derived); re-enable a deactivated login.

statistics.py — dashboard aggregates (prefix /api)

Endpoint	Description
GET /attendance/stats/overview	Totals by status, overall and per class (statusDistribution, classWiseStats).
GET /marks/stats/overview	Overall average, grade spread, and per-exam performance.
GET /fees/stats/overview	Collection totals, paid/partial/unpaid counts, splits by fee type and class.
GET /{resource}/stats/overview	Row count for every other resource (summary tiles).

reactivation.py — restoring soft-deleted records (prefix /api/reactivation)

Endpoint	Description
GET /deactivated	Every deactivated class, student and user, plus the counts the admin modal shows.
POST /class/{id}/[students], /student/{id}	Reactivate a class (optionally its students), or one student and their login.

timetable.py — schedule, calendar, exams, holidays (prefix /api/timemanagement)

Endpoint	Description
GET /calendar-week/{class}/{year}/{offset}	One week of the grid for a class: slots, periods, day types.
GET /student- teacher-calendar-week/...	The same week via the student's class, or narrowed to a teacher's periods.
GET /schedule class-schedule teacher-schedule	The recurring weekly timetable: all, per class, or per teacher.
POST PUT DELETE /schedule/{id}	Add, change or remove one period of the weekly timetable.
GET /calendar/{year}, /calendar-slots/{class}/{date}	The calendar grid for a year; one class's plan for one date.
GET POST /exams, /holidays; upcoming-*/{class all}	List or schedule exams and holidays; a date range expands per class.
PUT DELETE /exam/{id}, /holiday/{id}	Edit one, or cancel it back to a working day. (/api/calendar mirrors this.)

5. End-to-End Flow, Demo Data & Running It

Login, step by step

- User submits a role login form (e.g. StudentLogin.js).
- AuthContext.login() calls apiServiceSingleton, which POSTs /api/auth/login.
- auth.py verifies the password against the PBKDF2 hash in the users table, then calls create_token().
- The token + user profile (joined from students/teachers if applicable) come back as JSON.
- AuthContext stores the token in localStorage (per username) and sessionStorage (this tab), and sets it on apiServiceSingleton.
- AuthGuard sees isAuthenticated=true and the right role, and renders the role's dashboard.
- Each dashboard page calls apiServiceSingleton.getX(), which attaches the bearer token and hits the matching /api/... route in resources.py.
- resources.py builds a SQLAlchemy select() on the matching model, serialises the rows to camelCase, and the JSON is rendered as tables/charts in the page component.

Demo data (seeded automatically on first run)

Both backend/app/database/seed.py (SQLite) and database/postgresql_starter.sql (PostgreSQL) seed the identical dataset: 3 classes, 3 teachers, 12 students (11 active + 1 inactive), 5 subjects, a full school week of attendance, two exams of marks and two fee types per active student, one syllabus unit per class/subject, a nine-period weekly timetable, and a calendar of working days, holidays and exams generated around today — enough for every dashboard and timetable screen to render real data.

Role	Login	Password
Admin	admin	admin123
Teacher	teacher1, teacher2, teacher3	teacher123
Student	student1 ... student12	student123

Running it locally

```
# Terminal 1 – backend
cd backend
python -m pip install -r requirements.txt
python main.py # http://localhost:8080, auto-creates + seeds SQLite

# Terminal 2 – frontend
cd frontend
npm install
npm start # http://localhost:3000
```

Switching to PostgreSQL

Set DATABASE_URL to a postgresql+psycopg://... connection string (see backend/.env.example), then run database/postgresql_starter.sql once against that database with psql. The backend code is unchanged: the ORM declares lowercase column names, which is what Postgres expects and what SQLite matches case-insensitively.